

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Título del Proyecto:

Motor de álgebra lineal optimizado en ARM64

PONDERACIÓN: 30 pts

 **Tiempo estimado: 50 hrs**

Índice

1. MARCO FORMATIVO	3
1.1. Valor	3
1.2. Competencia(s)	3
1.3. Objetivo SMART	3
2. Resumen Ejecutivo	4
3. Enunciado del Proyecto	4
3.1 Descripción del problema a resolver	4
3.2 Alcance del proyecto	4
3.3 Entregables	5
4. Material de apoyo	7
5. Metodología	7
6. Recursos y herramientas a utilizar	8
7. Desarrollo de Habilidades Blandas	8
8. Cronograma	9
9. Rúbrica de Calificación	10
9.1 Requisitos para optar a la calificación	10
9.2 Resumen de Puntuaciones	12
9.5 Comentarios Generales	15
Detalle de la Calificación	15

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Disciplina	Al aprender un nuevo tema y un nuevo lenguaje de programación, la disciplina es muy importante para poder investigar a fondo las bases del conocimiento y no perder de vista el objetivo que es adquirir conocimiento sobre los procesadores y su lenguaje ensamblador.

1.2. Competencia(s)

Indique la competencia general del curso y la competencia específica trabajada en esta lectura.

Tipo de Competencia	
Competencia General	El estudiante comprende e integra los fundamentos de la arquitectura ARM, el funcionamiento de programas en lenguaje ensamblador y la interacción con el hardware mediante el análisis de instrucciones, estructuras de control y manipulación de memoria, para desarrollar soluciones óptimas en sistemas embebidos y entornos IoT
Competencia Específica	Comprende el funcionamiento de la arquitectura ARM, la interacción del hardware con el lenguaje a bajo nivel analizando instrucciones, estructuras de control y manejo de pines multipropósito permitiéndole desarrollar sistemas embebidos y entornos IoT

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Los estudiantes serán capaces de realizar cálculos de álgebra lineal a través de una solución de software desarrollada a bajo nivel.
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	Lograr el correcto funcionamiento de todas las funciones y los cálculos correctos debidamente identificados a través de consola,
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Utilizando el libro de texto del curso magistral, un editor de código como visual studio, qemu para la emulación de entorno ARM y linux para la utilización de librerías de compilación cruzada.

Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	Desarrollo de una herramienta eficiente que pueda ser utilizada en cursos del área matemática, desarrollando su pensamiento lógico, abstracto y crítico.
A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	Cumplir este aprendizaje en un plazo máximo de 7 semanas.

2. Resumen Ejecutivo

Este proyecto, desarrollado en el marco del curso de Arquitectura de Computadores y Ensambladores, aborda el desarrollo de una herramienta eficiente para realizar cálculos de álgebra lineal, fundamentales en diversas áreas de la ingeniería y la ciencia de datos. El problema identificado es la dificultad que enfrentan los estudiantes y profesionales al resolver manualmente operaciones complejas con matrices, como la transposición, el método de Gauss y la resolución de sistemas de ecuaciones lineales. Como solución, se propone el desarrollo de un motor de álgebra lineal capaz de automatizar estos procesos y ofrecer resultados precisos y rápidos. En este proyecto se pondrán en práctica competencias de programación estructurada, lógica matemática, optimización algorítmica y análisis numérico, fortaleciendo la capacidad para diseñar soluciones computacionales a problemas matemáticos reales.

3. Enunciado del Proyecto

3.1. Descripción del problema a resolver

Aprovechando la eficiencia de los procesadores ARM y la programación a bajo nivel, se solicita el desarrollo de una herramienta que permita realizar operaciones de álgebra lineal sobre matrices, evitando el cálculo manual y promoviendo la comprensión del manejo de memoria, estructuras de datos y algoritmos en ensamblador.

El sistema deberá permitir al usuario crear, almacenar, visualizar y operar matrices mediante un menú interactivo en consola, implementando cada funcionalidad directamente en lenguaje ensamblador ARM64.

Las matrices se consideran estructuras bidimensionales organizadas en filas y columnas, y deberán almacenarse en memoria de forma contigua.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Se almacena en memoria como un bloque contiguo de memoria [1, 2, 3, 4, 5, 6] donde cada número es un valor entero.

INGRESO DE DATOS

Se debe ingresar el número de filas y el número de columnas que tendrá la matriz a trabajar

Ingrese número de filas: 3

Ingrese número de columnas: 3

Luego se deberá de pedir el valor de cada celda con el siguiente formato de texto: $a[0][0]$ = valor.

Internamente el programa deberá de reservar memoria dinámica para $m \times n$ elementos, cargar los valores en registros o memoria secuencial, permitir el acceso al elemento $A[i][j]$ con la función row-major.

Luego de esto, se deberá de acceder a un menú en donde se podrán seleccionar las siguientes funcionalidades descritas.

FUNCIONALIDAD MATRIZ IDENTIDAD

Se deberá de transformar la matriz ingresada en una matriz identidad, para esto debe ser una matriz cuadrada que debe ser ingresada, cualquier matriz que no sea cuadrada, debera de mostrar un error en pantalla. La matriz identidad es donde todos los elementos de la diagonal principal son 1 y el resto son 0. Ejemplo:

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

FUNCIONALIDAD MATRIZ TRANSPUESTA

La matriz transpuesta consiste en intercambiar las filas por columnas de una matriz.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \Rightarrow A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

FUNCIONALIDAD MÉTODO DE GAUSS

El método de Gauss transforma una matriz en una forma triangular superior mediante operaciones de filas, sin alterar las soluciones del sistema lineal asociado.

$$\begin{bmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{bmatrix} \Rightarrow \begin{bmatrix} 2 & 1 & 1 \\ 0 & -8 & -2 \\ 0 & 0 & 1 \end{bmatrix}$$

FUNCIONALIDAD GAUSS-JORDAN

El método Gauss-Jordan extiende el anterior hasta convertir la matriz en forma reducida por filas donde la diagonal principal tienen un valor de 1 y todos los demás valores tienen 0.

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

FUNCIONALIDAD MATRIZ INVERSA

La matriz inversa es aquella que cumple con la regla de $A * A^{-1} = I$, donde I es la matriz identidad, esta funcionalidad se podrá ejecutar si y sólo si, la matriz es cuadrada.

$$A \times A^{-1} = I$$

FUNCIONALIDAD ARITMÉTICA

En esta sección se deberá desplegar un submenú que permita hacer las siguientes operaciones matemáticas entre matrices:

- Suma
- Resta
- Multiplicación (Punto y Cruz)
- División

Tomando en cuenta todas las restricciones de estas operaciones.

FUNCIONALIDAD DETERMINANTE

Calcular el determinante de la matriz en cuestión aplicando los parámetros definidos por las restricciones para el cálculo del determinante.

3.2. Alcance del proyecto.

- La aplicación debe permitir el cálculo de diferentes operaciones referentes a el álgebra lineal desarrollando algoritmos de forma eficiente, y con un alto rendimiento.

3.2.1. Requerimientos Tecnicos

- Los estudiantes deberán de utilizar el lenguaje ensamblador ARM de 64 bits, emulado en el entorno de máquinas virtual de qemu en un sistema operativo como linux utilizando las herramientas y dependendencias de este.

3.3 Entregables

Tipo	Descripción
Prototipo	Desarrollo de una versión funcional del proyecto que realice diferentes operaciones matriciales.
Documentación Técnica	Incluye manual técnico con todos los procedimientos desarrollados en el ensamblador y la lógica detrás de los cálculos sobre el álgebra lineal.
Código Fuente	El código completo del proyecto, con una estructura clara y comentada , que refleja las buenas prácticas de programación y uso de control de versiones.
Diagrama de Flujo	Representación gráfica del flujo de trabajo o funcionamiento del sistema, que permite comprender el recorrido de los datos o acciones dentro del proyecto.

4. Material de apoyo

- 4.1. ARM Architecture Reference Manual for A-profile Architecture (ARMv8-A)
<https://developer.arm.com/documentation/ddi0487/latest>
- 4.2. ARM Cortex-A Series Programmer's Guide for ARMv8-A
<https://developer.arm.com/documentation/den0024>
- 4.3. ARM GNU Toolchain Documentation (Assembler, Linker, and Debugger)
<https://gcc.gnu.org/onlinedocs/>
- 4.4. "Learning ARM64 Assembly" — Azeria Labs <https://azeria-labs.com>
- 4.5. "ARM64 Assembly Programming for Linux" — Tutorialspoint
https://www.tutorialspoint.com/assembly_programming/

5. Metodología

- 5.1. Se deberá de seguir una metodología SCRUM, en donde se presentan avances significativos cada semana como evidencia del desarrollo de su proyecto.
- 5.2. Investigación preliminar: Los estudiantes deben realizar una investigación sobre el cálculo a desarrollar, sus restricciones y el debido proceso que debe aplicarse.
- 5.3. Diseño del sistema: Crear diagramas de flujo o esquemas de la aplicación que representen cómo será el comportamiento de la información dentro de la aplicación
- 5.4. Desarrollo: Los estudiantes deberán implementar su solución en fases, desarrollando funciones de cálculo de forma semanal según las funcionalidades del programa.
- 5.5. Pruebas y ajustes Se realizarán pruebas para validar que la función desarrollada en cuestión funcione correctamente de forma general.

6. Recursos y herramientas a utilizar

Listado de materiales que los estudiantes deberán usar o investigar:

- Software: Qemu, Aarch64, Draw.io
- Plataformas: UEDI, GitHub, Visual Studio Code
- Lecturas recomendadas: documentación oficial, videos sobre cálculos de álgebra lineal

7. Desarrollo de Habilidades Blandas

7.1 Proyectos Individuales

El trabajo individual es clave para que los estudiantes se muestren a sí mismos la capacidad que tienen frente a la resolución de problemas donde es fundamental la aplicación de su lógica de programación. Creando un ciclo de retroalimentación en donde a través de estos retos, entrenan su lógica y se demuestran la capacidad analítica que tienen para llevar a cabo algoritmos complejos.

7.1.1 Comunicación Efectiva

Se realizarán debates en clase en donde los estudiantes puedan exponer sus dudas e inquietudes y aclarar las ideas de cómo llevar a cabo la solución de un problema complejo.

7.1.2 Resolución de Conflictos

A lo largo del proyecto los estudiantes pueden encontrarse a estancamientos mentales, o a la incomprensión de algunos conflictos, por lo que es primordial, que cada estudiante desarrolle su propia metodología.

8. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	2026-03-21	2026-03-21
Elaboración	2026-03-21	2026-05-01
Calificación	2026-05-02	2026-05-02

9. Rúbrica de Calificación

9.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Si/No)
Cumplimiento de la tecnología establecida	El desarrollo debe haberse realizado en el lenguaje ensamblador ARM64.	
Gestión y entregas del proyecto	Se deben haber realizado commits de forma regular durante el desarrollo del proyecto (Mínimo un commit Semanal)	
Documentación	Se debe entregar documentación técnica completa.	
Pruebas y funcionalidad mínima	El sistema debe ser funcional y cumplir con al menos los requisitos mínimos. Debe haber evidencia de pruebas realizadas y reporte de resultados.	
El auxiliar tiene acceso al repositorio	El auxiliar tiene que ser colaborador del repositorio del proyecto para poder verificar avances, commits, etc. Nombre del Repositorio: ARQUI1_1S2026_P2	

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Ingreso de datos	10				
1.1	Entrada de datos	4	Descripción	Solicita correctamente filas, columnas y valores desde consola y el estudiante comprende el flujo de entrada	Entrada incompleta, incorrecta o no comprende cómo se obtienen los datos	
			Rango del punteo	4-3	/ 2-0	
1.2	Almacenamiento	3	Descripción	Usa memoria contigua correctamente (row-major) y puede explicar el acceso $A[i][j]$	Datos mal almacenados o no comprende acceso en memoria	

			Rango del punteo	3–2	1–0	
1.3	Validación	3	Descripción	Maneja errores (dimensiones, matriz no cuadrada) y comprende por qué son necesarios	No valida o no comprende las restricciones	
			Rango del punteo	3–2	1–0	
	Sub-Total de Puntos	10				
2	Funcionalidades de álgebra lineal	35				
2.1	Matriz identidad	5	Descripción	Genera correctamente, valida matriz cuadrada y explica lógica	Resultado incorrecto o no comprende la diagonal	
			Rango del punteo	5–3	2–0	

2.2	Transpuesta	5	Descripción	Intercambia filas/columnas correctamente y entiende el proceso	Lógica incorrecta o sin comprensión	
			Rango del punteo	5-3	2-0	
2.3	Método de Gauss	7	Descripción	Reduce correctamente y explica operaciones de fila	Procedimiento incorrecto o no comprende el método	
			Rango del punteo	7-5	4-0	
2.4	Gauss-Jordan	6	Descripción	Lleva a forma reducida y entiende pivotes y normalización	No logra forma correcta o no entiende el proceso	
			Rango del punteo	6-4	3-0	
2.5	Matriz inversa	6	Descripción	Calcula correctamente, valida y explica el proceso	Error en cálculo o sin comprensión	
			Rango del punteo	6-4	6-4	

2.6	Determinante	6	Descripción	Calcula correctamente y comprende el método aplicado	Resultado incorrecto o sin comprensión	
			Rango del punteo	6–4	3–0	
	Sub-Total de Puntos	35				
3	Operaciones aritméticas	15				
3.1	Suma	3	Descripción	Opera correctamente y valida dimensiones, comprende la operación	No valida o resultado incorrecto	
			Rango del punteo	3–2	1–0	
3.2	Resta	3	Descripción	Correcta y consistente, con comprensión	Resultado incorrecto o sin comprensión	
			Rango del punteo	3–2	1–0	

3.2	Multiplicación	5	Descripción	Implementa correctamente (filas x columnas) y entiende el proceso	Lógica incorrecta o no comprende	
			Rango del punteo	5–3	2–0	
3.2	División	4	Descripción	Implementación válida o justificación técnica clara	No implementa o sin justificación	
			Rango del punteo	4–3	2–0	
	Sub-Total de Puntos	15				
4	Uso de ensamblador y arquitectura	15				
4.1	Uso de registros	5	Descripción	Uso eficiente y puede explicar propósito de registros	Uso incorrecto o sin comprensión	
			Rango del punteo	5–3	2–0	

4.1	Acceso a memoria	5	Descripción	Accede correctamente y entiende direccionamiento	Accesos incorrectos o no comprende	
			Rango del punteo	5-3	2-0	
4.1	Control de flujo	5	Descripción	Bucles y condiciones correctos y comprensibles	Lógica incorrecta o sin comprensión	
			Rango del punteo	5-3	2-0	
	Sub-Total de Puntos	15				
5	Diseño del sistema	10				
5.1	Organización	10	Descripción	Código estructurado, modular y comprensible	Código desordenado	
			Rango del punteo	3-2	1-0	

5.2	Flujo	10	Descripción	Menú claro y coherente, comprende el flujo	Flujo confuso	
			Rango del punteo	3–2	1–0	
5.3	Diseño eficiente	10	Descripción	Evita sobreingeniería (uso de pocas matrices, reutilización) y lo justifica	Uso excesivo de estructuras (ej: múltiples matrices innecesarias)	
			Rango del punteo	4–3	2–0	
	Sub-Total de Puntos	10				
6	Optimización y eficiencia	5				
6.1	Uso de memoria	2	Descripción	Uso eficiente sin duplicación innecesaria	Uso excesivo de memoria	
			Rango del punteo	2–2	1–0	

6.2	Reutilización	2	Descripción	Reutiliza matrices (A, B, resultado)	No reutiliza datos	
			Rango del punteo	2-2	1-0	
6.3	Complejidad	1	Descripción	Solución simple y clara	Solución innecesariamente compleja	
			Rango del punteo	1-1	0-0	
	Sub-Total de Puntos	5				
7	Código y buenas prácticas	5				
7.1	Legibilidad	2	Descripción	Código claro y comentado	Difícil de entender	
			Rango del punteo	2-2	1-0	
7.2	Organización	2	Descripción	Orden y consistencia	Código desorganizado	

			Rango del punteo	2-2	1-0	
7.3	Nombres	1	Descripción	Uso claro de etiquetas/registros	Nombres confusos	
			Rango del punteo	1-1	0-0	
	Sub-Total de Puntos	5				
8	Documentación y diagramas	5				
8.1	Documentación	3	Descripción	Código claro y comentado	Difícil de entender	
			Rango del punteo	3-2	1-0	
8.2	Diagramas	2	Descripción	Orden y consistencia	Código desorganizado	

			Rango del punteo	2–2	1–0	
	Sub-Total de Puntos	5				
	TOTAL	100				

9.2 Comentarios Generales

Durante la evaluación, el estudiante podrá ser consultado sobre el funcionamiento de su código.

Es requisito demostrar comprensión clara de la lógica implementada, incluyendo el manejo de memoria, estructuras y algoritmos utilizados.

La evaluación podrá incluir la explicación de fragmentos específicos del código, así como la justificación de decisiones de diseño y la descripción del flujo de ejecución del programa.

En caso de que el estudiante no pueda explicar de forma coherente y consistente su propia implementación, se considerará que no existe evidencia suficiente de autoría.

Esta situación afectará directamente la calificación del proyecto, pudiendo implicar la reducción significativa de la nota o la anulación parcial o total del mismo.

